Elevator System Simulation – Take-Home Project

Background

Modern elevator systems go far beyond a single elevator serving every floor. They feature elevator banks, sky lobbies, express elevators, and intelligent scheduling to minimize wait and travel times.

In this project, you'll design a simplified simulation of an intelligent elevator system. You'll implement a discrete-time model of elevators serving requests in a building, simulating how real-time requests are handled efficiently across multiple elevators.

Objectives

Your system should aim to achieve the following:

1. **Serve all requests eventually.** No passenger should wait indefinitely to be picked up or dropped off.
2. **Minimize total time per passenger**, where: `total_time = wait_time + travel_time`
3. **Honor elevator constraints**, including capacity and direction logic.

System Type

This system models a modern Destination Dispatch elevator system, where passengers specify both origin and destination in advance, allowing the controller to make optimized routing decisions where:

- A passenger submits both their origin and destination floor at the time of request.
- The system immediately assigns them to a specific elevator.
- Once assigned, the passenger cannot modify their destination.

Time Modeling

Time will be modeled in **discrete units**:

- One time unit = one floor of travel (up or down).
- You do not need to sync to real clock time.
- Your simulation must tick forward **one time unit at a time**, even if input requests skip ahead.

Requirements

Build a Python model of an elevator system with the following features:

- Configurable number of elevators (e.g., 1–10)
- Configurable number of floors (`n`)
- Configurable maximum passengers per elevator

Implement a **scheduler algorithm** of your choice that respects the goals outlined above.

Input

Write a function that accepts a list of elevator requests with the following fields:

```
time,id,source,dest
0,passenger1,1,51
0,passenger2,1,37
10,passenger3,20,1
```

Each row represents a single request:

- `time`: Integer time step when request is made
- `id`: Unique passenger ID
- `source`: Origin floor
- `dest`: Destination floor

Your simulation should **not peek ahead** in the request list beyond the current time.

Output

1. **Elevator Positions Log**
    - For every time step (starting from 0), log the location of all elevators to a file.
    - Format: one row per timestamp, showing elevator positions at that time.
2. **Passenger Summary Statistics**
    - Upon simulation completion, output:
        - Minimum, maximum, and average **wait times** and **total times**
        - Any other notable observations about time distributions or system behavior

---

Bonus (Optional)

- Implement different elevator algorithms (e.g., round robin, nearest car, zone-based)
- Simulate "express elevators" that skip certain floors
- Explore trade-offs between fairness (serving all passengers quickly) and efficiency (optimizing for majority flow)

Deliverables

Please submit:

- A public GitHub repository containing your Python code
- A `README.md` with:
    - How to run your code
    - How long you spent on the project
    - Any assumptions, simplifications, or trade-offs you made
    - What you'd improve with more time

Presentation

Be prepared to present your simulation and discuss your decisions in a follow-up meeting. If you'd like to use any visualizations or statistics in your walkthrough, feel free to include them in your repo or bring them to your presentation.

Thanks for taking the time to work on this — we look forward to seeing your simulation in action!